SOFTWARE REQUIREMENTS SPECIFICATION

Book Stock Management System

Version 1.0 | *Prepared: 13 October 2026*

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the expected requirements for the Book Stock Management System, a Java-based web application intended to maintain book records, categories, stock information, and book issue/return records. It provides a reference for implementation, testing, and future maintenance.

1.2 Scope

The system is intended to provide a centralized web interface for managing book inventory. The described scope includes viewing dashboard information, managing books and categories, searching for books, viewing stock, and recording circulation transactions, to the extent these features are implemented in the current application.

1.3 Intended Audience

This document is intended for the project developer, project guide/evaluator, testers, and future maintainers.

1.4 Definitions and Acronyms

- SRS — Software Requirements Specification
- JSP — JavaServer Pages
- JDBC — Java Database Connectivity
- DBMS — Database Management System
- UI — User Interface

2. Overall Description

2.1 Product Perspective

The application is a browser-accessed Java web application. JSP pages present the interface, Java Servlets process HTTP requests, JDBC communicates with MySQL, and the database stores book, category, and circulation information.

2.2 Product Functions

- Display dashboard information available in the application.
- Create, view, update, and search book records.
- Maintain book categories and associate books with categories.
- Display stock quantities and availability information.
- Record book issue and return transactions, if supported by the current implementation.
- Store and retrieve application records from MySQL.

2.3 User Classes

The SRS assumes an operator who uses the application's book-management functions. Separate administrator, librarian, student, or member roles are not assumed unless role-based access has been implemented.

2.4 Operating Environment

- Operating system: Windows or another OS supported by the installed Java and MySQL versions.
- IDE/development environment: Eclipse IDE.
- Application server: Apache Tomcat 10.1.
- Backend: Java Servlets and JSP using the Jakarta Servlet API compatible with Tomcat 10.1.
- Database: MySQL Server.
- Database connectivity: JDBC with MySQL Connector/J.
- Client: modern web browser.

2.5 Constraints and Assumptions

- The MySQL service and required database must be available for database-backed functions.
- The project must use a compatible JDK, Tomcat version, and Jakarta Servlet API.
- The database schema must match the SQL queries used by the application.
- Features listed as optional or conditional must be verified against the current source code.
- Database credentials must be configured securely and must not be committed to a public repository.

3. Functional Requirements

FR-01: Dashboard

The system shall display the dashboard page and any summary information supported by the application.

FR-02: View Books

The system shall retrieve and display book records stored in the database.

FR-03: Add Book

The system shall allow an authorized operator, where authorization is implemented, to submit book details and save a valid record.

FR-04: Update Book

The system shall allow an existing book record to be updated.

FR-05: Search Books

The system shall allow users to search for books using the search criteria supported by the interface.

FR-06: Manage Categories

The system shall retrieve and maintain category information according to the implemented category operations.

FR-07: Associate Category

The system shall associate a book with a valid category when the database schema supports this relationship.

FR-08: View Stock

The system shall display book quantity and stock information stored in the database.

FR-09: Issue Book

If implemented, the system shall record an issue transaction and apply the configured stock rules.

FR-10: Return Book

If implemented, the system shall record a return transaction and apply the configured stock rules.

FR-11: Persist Data

The system shall store and retrieve supported records using the configured MySQL database.

FR-12: Validate Input

The system should validate required input and reject or safely handle invalid values.

FR-13: Handle Errors

The system should handle database and application errors without exposing credentials or sensitive configuration.

4. External Interface Requirements

4.1 User Interface

The user interface shall be presented through JSP/HTML pages and associated CSS. Forms should have clear labels, tables should present records in a readable format, and navigation should make implemented features easy to locate. Exact screen layouts are determined by the existing UI.

4.2 Software Interfaces

- Java application components communicate with MySQL through JDBC.
- MySQL Connector/J provides the MySQL JDBC driver.
- Apache Tomcat serves the web application and processes supported Servlet requests.
- JSP pages render application views in the user's browser.

4.3 Communication Interfaces

The browser communicates with the Tomcat-hosted application over HTTP or HTTPS according to the local or deployed server configuration. The application server communicates with MySQL using the configured database connection.

5. Non-Functional Requirements

NFR-01: Usability

The interface should be understandable to a user familiar with basic web forms and tables.

NFR-02: Performance

Common read and update operations should complete within a reasonable time under the expected project workload.

NFR-03: Reliability

Database operations should report failures clearly and avoid presenting an unsuccessful operation as successful.

NFR-04: Data Integrity

Book, category, and circulation data should remain consistent with the database schema and transaction rules.

NFR-05: Maintainability

Code should be organized into appropriate packages and web resources so that modules can be maintained independently.

NFR-06: Security

Credentials must not be hard-coded into public source code; SQL statements using user input should use parameterized queries where applicable.

NFR-07: Compatibility

The application should run with the configured Java version, Tomcat 10.1, compatible Jakarta Servlet APIs, and supported MySQL Connector/J.

6. Data Requirements

The known database name is book_stock_db. The project context identifies three principal tables: categories, books, and circulation. The exact columns, data types, keys, constraints, and transaction rules must be taken from the project's actual SQL schema.

6.1 Categories

Stores category information used to organize books. Expected conceptual data includes a category identifier and category name.

6.2 Books

Stores book information and stock quantities. Expected conceptual data includes a book identifier, title, category reference, and quantity, subject to the actual schema.

6.3 Circulation

Stores issue/return information and a reference to the relevant book, subject to the actual schema.

6.4 Data Validation

- Identifiers should reference existing records where foreign-key relationships are defined.
- Required fields should not be empty.
- Stock quantities should not be negative unless a specifically documented business rule requires otherwise.
- Issue and return operations should follow the rules implemented by the application.

7. High-Level System Architecture

- Presentation layer: JSP, HTML, CSS.
- Request-processing layer: Java Servlets in the com.library.servlet package.
- Database connectivity layer: DBConnection/JDBC in the com.library.db package.
- Persistence layer: MySQL database book_stock_db.

Browser → JSP Pages → Java Servlets → JDBC / DBConnection → MySQL

8. Acceptance Criteria

- The project deploys successfully to the configured Tomcat server.
- The application can connect to the configured MySQL database.
- Implemented book and category operations return expected results.
- Search and stock views display data consistent with the database.

- Issue and return workflows pass their defined tests if these features are implemented.
- Invalid input and database errors are handled appropriately.
- No real database password or other secret is exposed in the public repository.

9. Testing Requirements

Testing should include functional tests for each implemented operation, database connectivity tests, validation and error-handling tests, and integration tests across JSP pages, Servlets, and MySQL. Record actual pass/fail results in the project's testing report.

Test Area	Expected Result	Actual Result
Database connection	Connection succeeds with valid configuration	[Record result]
Book management	Valid book operations work correctly	[Record result]
Category management	Category information is handled correctly	[Record result]
Search and stock	Displayed results match stored data	[Record result]
Issue/return	Transactions follow implemented stock rules	[Record result]
Error handling	Invalid operations are safely handled	[Record result]

10. Limitations and Future Enhancements

Potential enhancements include authentication and role-based access, member management, due-date tracking, overdue fine calculation, low-stock alerts, report export, dashboard analytics, and automated testing. These are proposed enhancements and are not claims about the current implementation.

11. References and Project Information

Project: Book Stock Management System

Repository: https://github.com/khushalvaishnav020140-cmd/BookStockManagement2

Technology stack: Java, JSP, Servlets, JDBC, MySQL, Apache Tomcat 10.1, Eclipse IDE.

Prepared by: [Add your name]

Project guide: [Add guide name if applicable]

Document status: Draft — verify requirements against the actual source code and database schema before final submission.